

A Resource-Efficient Hardware Accelerator for Lossless Image Compression Based on the Walsh–Hadamard Transform

1st Marco Zolla Salazar
Maestría en Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica
mzolla@estudiantec.cr

2nd Luis Diego Martinez Jimenez
Maestría en Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica
diego_mj22@estudiantec.cr

3rd Simón Fallas Villalobos
Maestría en Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica
simonfallasv@estudiantec.cr

4th Marcelo Mata Solano
Maestría en Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica
m.mata.3@estudiantec.cr

5th Brandon Fabricio Rivera Hidalgo
Maestría en Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Costa Rica
b.rivera.2@estudiantec.cr

Abstract—This work presents a resource-efficient hardware accelerator for an integer and reversible Walsh–Hadamard Transform intended for lossless image compression. The proposed architecture uses a lifting-based formulation composed only of additions, subtractions, and arithmetic shifts, avoiding DSP blocks while preserving exact reconstruction. The transform core was implemented using Vitis HLS and evaluated against a bit-accurate reference model. Source-level functional verification showed exact agreement between the reference model and the synthesizable forward and inverse cores over more than 100,000 random blocks each, and exact reconstruction through the cascaded forward and inverse cores was confirmed over 400,000 cases; no C/RTL cosimulation was performed. For 8-bit input data, a 16-bit datapath was sufficient without overflow. Entropy experiments showed reductions of approximately 20% for spatially correlated natural-like data and up to 92% for highly structured images. Post-route implementation achieved 339.21 MHz using 2404 LUTs, 2599 flip-flops, four BRAM tiles, and zero DSP blocks. These results demonstrate the feasibility of a high-throughput multiplier-free WHT accelerator as a transform stage within a complete lossless compression pipeline.

Index Terms—Discrete transforms, Image coding, Data compression, Field programmable gate arrays, Hardware acceleration, High level synthesis

I. INTRODUCTION

Modern high-frame-rate camera systems, such as those deployed in intelligent vehicles and satellite imaging, generate massive volumes of data that must be processed and stored with minimal latency. In many of these edge-computing scenarios, lossless compression is essential because the uncertainty introduced by quantization noise in lossy systems is often inadmissible for maintaining system robustness or meeting legal requirements [7]. Implementing these codecs in real-time at the edge is a critical challenge, as embedded video processors must handle high-speed transform computations

while operating under strict limitations regarding hardware area, power consumption, and thermal dissipation [11].

Transform methods are a fundamental component of image compression because they decorrelate spatial data before encoding, concentrating information into a smaller number of coefficients. Among these, the Walsh-Hadamard Transform (WHT) is particularly attractive for hardware implementations due to its significant computational advantages over sinusoidal transforms like the DFT or DCT. Because the WHT kernel matrix consists entirely of integer values of +1 and -1, the transform can be implemented very efficiently using only additions and subtractions, thereby eliminating the need for costly and time-consuming multipliers [1], [4].

Despite its advantages, state-of-the-art literature reveals limitations in existing solutions, which are frequently optimized for lossy applications or general neural network acceleration rather than resource-efficient lossless compression. For example, some high-throughput WHT-based architectures are designed for signal processing but rely on 32-bit floating-point arithmetic, which is not ideal for exact, low-resource lossless operation [1]. Conversely, other lossless approaches, such as those based on the JPEG-LS standard, provide competitive compression but often do not leverage the unique hardware-friendly properties of the WHT [7].

In this paper, we propose a resource-efficient integer/reversible WHT accelerator specifically designed for low-latency lossless compression. Our main contributions are:

- An optimized integer/reversible WHT architecture that ensures mathematically exact data reconstruction for lossless applications.
- The development of a flexible design modeled at a high level using SystemC and HLS, allowing rapid design-space exploration and the generation of parallel-pipeline

layouts.

- A preliminary functional and post-route FPGA evaluation reporting timing, throughput, and resource-utilization results.

The remainder of this paper is organized as follows. Section II presents the background and related work. Section III describes the proposed architecture and its high-level implementation. Section IV reports the functional verification, entropy analysis, and post-route hardware results.

II. BACKGROUND AND RELATED WORK

The Walsh–Hadamard transform (WHT) decomposes a signal into Walsh basis functions whose values are restricted to ± 1 . For a signal of 2^n samples the transform is a multiplication by the $2^n \times 2^n$ Hadamard matrix, and since its entries are ± 1 it can be evaluated with additions and subtractions alone. The fast WHT (FWHT) lowers the arithmetic cost from $O(N^2)$ to $O(N \log_2 N)$ through Cooley–Tukey-style factorizations [1], and its parallel-pipeline hardware implementations follow processor structures first developed for the fast Fourier transform [14].

A conventional integer implementation of the WHT does not necessarily guarantee lossless reconstruction because its inverse requires normalization by the transform size N . This division may not be represented exactly using integer arithmetic, causing information loss during a forward–inverse transform cycle. Integer, reversible formulations remove this limitation. The unified lossless WHT (ULWHT) recovers the original data exactly while lowering the per-coefficient entropy of natural images [2], and a lossless two-dimensional WHT has also been derived [3]. For hardware there is a caveat: the one-dimensional lossless WHT still contains multiplications, and making it fully multiplier-free needs a more elaborate construction [3].

Two closely related solutions provide reproducible results with public code, and between them they cover both sides of the problem. One is an FPGA WHT accelerator: a configurable parallel-pipeline FWHT produced by high-level synthesis, with its HLS source openly released [1]. The other is a parallel lossless compressor for raw Bayer images on an FPGA-based high-speed camera that sustains 40.1 Gbit/s at 320 MHz on a Zynq UltraScale+ device, using adaptive Golomb–Rice coding and a public reference implementation [5]. One supplies the transform, the other a low-latency lossless pipeline; bringing the two together is the open problem.

Five further works address parts of the problem. In hardware, a multiplier-free WHT running at 162 MHz takes the place of the Photo Core Transform in JPEG XR [4], an FPGA JPEG 2000 accelerator performs on-board lossless compression [6], and an ANS-based coder reduces the complexity of JPEG-LS [7]. The integer lossless WHT formulations achieve bit-exact compression but stop at the algorithmic level, with no hardware realization [2], [3]. Earlier WHT processors based on distributed arithmetic [11], the sequency-ordered complex Hadamard transform [12], and a radix-2 Walsh–Hadamard–

Fourier pipeline [13] were built for high throughput and small area, drawing on the low complexity of the ± 1 kernel [8].

The WHT also appears outside compression, in image watermarking [15], ultrasonic coded excitation [9], and the simulation of quantum algorithms [10]. To the best of the authors’ knowledge, no recent FPGA accelerator combines a reversible lossless WHT, a multiplier-free implementation, and publicly reproducible results for high-frame-rate image processing. The reproducible accelerators cover either the transform [1] or the lossless coding [5], and the lossless integer WHT has stayed at the algorithmic level [2], [3]. Closing this gap is the aim of the present work.

III. PROPOSED SOLUTION

A. Study of Other Solutions in the State of the Art

An analysis of recent literature over the past ten years reveals that implementations oriented toward image compression bifurcate into approaches that partially cover the problem but lack an efficient architectural integration specifically oriented toward a multiplier-free design for lossless video streams.

1) *Hardware Accelerators for Transforms (Lossy Approach)*: Hardware accelerators for transforms oriented toward lossy approaches have demonstrated remarkable efficiency in the real-time domain. These architectures achieve high data processing throughput by utilizing parallel and pipelined logic structures (*parallel-pipeline*) [1], also proving the physical feasibility of operating at high clock frequencies, such as 162 MHz, completely omitting advanced DSP logic blocks [4]. Despite these strengths, these solutions present severe limitations when considered for lossless environments, as they are optimized for generic processing or traditional lossy schemes relying on 32-bit floating-point arithmetic [1]. Furthermore, using the conventional inverse transform introduces division operations for normalization that integer-based systems cannot represent exactly, injecting quantization noise that destroys the possibility of exact data reconstruction [2]. Consequently, while these designs represent a valuable metric baseline for evaluating hardware cost in terms of area and speed, they are not directly suitable for strictly lossless compression applications.

2) *Lossless Hardware Compressors (Complete Pipelines)*: On the other hand, developments focused on complete lossless compression pipelines offer robust, low-latency alternatives essential for edge computing. Solutions such as parallel compressors for raw Bayer images implemented on FPGAs [5] and structural optimizations applied to the JPEG-LS standard under advanced coding architectures [7] can sustain massive data transfer rates in real-time. However, the main deficiency of this approach is that optimization efforts are directly concentrated on the entropy coding and packaging stages, omitting or completely replacing the spatial decorrelation phase based on discrete transforms such as the WHT [7]. This omission makes them highly viable for general-purpose compression at the edge, but not directly applicable when the architectural design goal is to maximize datapath efficiency by exploiting

a multiplier-free transformation core composed exclusively of +1 and -1 coefficients [4].

3) *Reversible WHT Algorithmic Formulations*: Finally, proposals focused on the theoretical formulations of the reversible transform resolve the mathematical limitations of information loss in the inverse process. Mathematical models such as the unified lossless Walsh-Hadamard transform (ULWHT) and its two-dimensional derivations formally demonstrate that it is viable to recover data exactly bit-by-bit, significantly reducing the entropy per coefficient without resorting to complex multiplication operations [2], [3]. The major weakness of this approach lies in its purely abstract nature, given that these formulations have remained strictly in the algorithmic domain and lack a tangible realization or microarchitectural mapping in hardware for embedded systems [2]. Thus, although these formulations are suitable as numerical reference models, they require hardware-oriented architectural redesign before direct FPGA implementation.

B. Observed Challenges

Based on the critical study of the state of the art, the fundamental architectural challenges that justify the contribution of this research are identified and outlined. The first challenge consists of properly delimiting the functional scope of the accelerator. Since the WHT transform acts solely as a spatial decorrelator and not as an autonomous codec, it is imperative to narrow the architecture to a dedicated transform core accelerator, structured in such a way that it can be seamlessly integrated into a larger lossless pipeline without diverting logic efforts toward complex entropy coding. The second technical challenge lies in performing a direct mapping from the abstract algorithmic domain to a real physical architecture with zero use of DSP blocks. Although the literature describes theoretical reversible and multiplier-free versions [3], their implementation through high-level synthesis (SystemC/HLS) requires optimizing subexpression reuse and designing aggressive pipelining to mitigate FPGA resource consumption.

As a third challenge, there is a need to strongly justify the choice of the WHT over competing integer transforms such as the DCT or wavelets. Because the WHT exhibits inferior performance in lossy compression quality [5], the proposed architecture must be empirically evaluated in terms of logic area cost and maximum operating frequency in a lossless environment. The fourth challenge focuses on ensuring real-time operational performance for high-frame-rate video streams, which forces the design of a datapath with a highly optimized transaction-level data flow that minimizes stall cycles under the strict thermal and power constraints of embedded systems [11]. Finally, the fifth challenge corresponds to the verification of bit-exact reproducibility. To ensure identical reconstruction of the original data, a bit-accurate SystemC reference model was developed and used as the verification oracle for deterministic and randomized comparisons against the HLS implementation.

C. Scope and Design Philosophy

Based on the analysis presented in Section II, this work addresses the limited availability of recent FPGA accelerators that combine a reversible WHT, multiplier-free arithmetic, and reproducible implementation results. Therefore, this work focuses on the development of an accelerator dedicated exclusively to the execution of the transform. The proposed system performs an integer and reversible Walsh-Hadamard Transform on pixel blocks obtained from a high-speed video stream. The output consists of transform coefficients that can subsequently be processed by an entropy coding stage, such as Golomb-Rice or ANS coding. This design boundary allows the evaluation of only the hardware associated with the transform itself, quantifying its efficiency in terms of resource utilization and performance without being influenced by the compression ratio of a complete codec.

D. High-Level Architecture

The proposed architecture consists of five main functional blocks, as illustrated in Fig. 1.

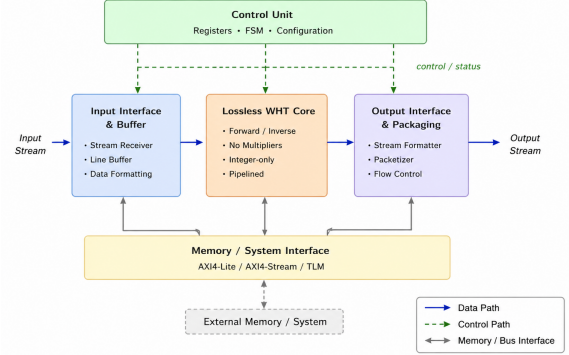


Fig. 1. High-level architecture of the proposed lossless WHT accelerator.

1) *Input Buffer and Interface (IF)*: This block receives pixels from the camera or memory subsystem and organizes them into transform blocks. The implemented architecture uses a block size of $N = 8$, selected as a practical compromise between hardware complexity, latency, and verification effort.

2) *Lossless WHT Core*: The computational core of the accelerator was designed in Vitis HLS utilizing a fully parallel, multiplier-free architecture. To satisfy the requirements of a lossless environment (zero quantization noise) while keeping DSP utilization at zero, the standard fast Walsh-Hadamard butterfly was replaced by an integer lifting-based scheme (S-Transform) [2].

The fundamental spatial decorrelation step between two pixels a and b is implemented purely through high-frequency differences ($d = a - b$) and low-frequency shifted averages ($s = a - (d \gg 1)$). This specific subexpression routing is synthesized by the HLS compiler completely into LUTs and Flip-Flops, bypassing any DSP slice instantiation.

To maximize datapath throughput for real-time video, the transform operations for a block size of $N = 8$ were flattened

into three combinational butterfly stages. The reported processing latency of 150 cycles is not due to the combinational datapath itself, but rather arises from the `m_axi` memory transactions required to read and write the data bursts. We enforce maximal hardware concurrency by applying the `#pragma HLS PIPELINE II=1` and `#pragma HLS UNROLL` directives. Additionally, variables were strictly constrained using HLS arbitrary precision integers (`ap_int<16>`), which prevents the datapath from unnecessarily routing 32-bit standard C++ integer bounds, ensuring a highly resource-efficient mapping on the Kria KV260 architecture.

Although aggressive loop unrolling and full spatial parallelism inherently increase the combinatorial logic footprint, these datapath design decisions are rigorously justified by the strict latency requirements of real-time video processing. By prioritizing throughput, the core achieves an initiation interval of 1, allowing it to process a new data block every clock cycle. The resulting hardware cost (approximately 2400 LUTs) is a worthwhile trade-off, as it completely avoids the instantiation of DSP slices while comfortably meeting the timing and throughput constraints. The complete HLS source code for the transform core is publicly available in our repository [16].

3) *Control Unit*: The control unit manages the overall operation of the accelerator. Although it does not directly perform the WHT computation, it coordinates when pixel blocks enter the system, when temporary data are stored in registers, and when results are forwarded to the next pipeline stage. Furthermore, it synchronizes processing to prevent data conflicts and enables architectural parallelism through control signals such as enable, valid, and ready.

4) *Output Interface and Packaging*: This module collects the transformed coefficients and packages them for transmission or subsequent entropy coding. Like the input buffer, it is part of the system-level integration.

5) *Memory Interface (AXI4/AXI4-Lite)*: This module manages data transfers between the accelerator and the external memory subsystem. The use of AXI4 and AXI4-Lite interfaces facilitates integration into FPGA-SoC platforms such as the Kria KV260 and Alveo U55C, allowing the accelerator to be incorporated into larger embedded processing systems.

The current implementation and evaluation focus on the WHT computational core. The control unit, input buffering, output packaging, and external-memory interfaces remain part of the proposed system-level integration and were not independently evaluated in the reported experiments.

E. Challenge–Solution Mapping

The proposed architecture aims to address the main challenges identified during the literature review. Table I summarizes how each challenge is considered within the proposed solution.

Overall, the proposed architecture provides a feasible FPGA implementation of a lossless Walsh-Hadamard Transform while addressing the main limitations identified in previous studies.

TABLE I
CHALLENGE–SOLUTION MAPPING

Challenge	Proposed Solution
D1 – Codec Scope	This work focuses only on the WHT stage. Entropy coding (e.g., Golomb-Rice, ANS) is considered a later processing stage, allowing the transform hardware to be evaluated independently.
D2 – DSP-Free FPGA Implementation	The WHT core uses only additions, subtractions, and arithmetic right shifts based on the lifting construction from [2], minimizing DSP usage and mapping entirely to LUTs and Flip-Flops.
D3 – WHT vs. DCT/Wavelets	The current work reports the timing and resource utilization of the proposed WHT core. A quantitative comparison with integer DCT and other WHT accelerators remains as future work.
D4 – Real-Time Throughput	A fully pipelined datapath was implemented with an initiation interval of one cycle, supporting high-throughput operation.
D5 – Bit-Exact Lossless Reconstruction	The reversible lifting-based formulation guarantees exact reconstruction. A bit-accurate reference model performs sample-by-sample comparisons against the forward and inverse HLS cores at the C/C++ source level; verification against the generated RTL is not covered and remains as future work.

IV. RESULTS AND ANALYSIS

This section presents the experimental methodology used to evaluate the proposed accelerator and to contrast it against the state of the art, covering the modeling and synthesis toolchain, the experiments and baselines, and the metrics collected for the comparison. In addition to entropy and bit-exactness checks, we report post-route FPGA metrics.

A. Experimental Setup and Toolchain

A bit-accurate reference model of the transform was implemented in C++ with a SystemC 2.3.4 transaction-level wrapper, compiled with g++ 13.3.0, and used as the verification oracle. The model reproduces the integer lifting butterfly of both synthesizable cores, forward and inverse. The register-transfer implementation was obtained through high-level synthesis with AMD Xilinx Vitis HLS and synthesized with Vivado, targeting the AMD Kria KV260 (Zynq UltraScale+ MPSoC, part `xck26-sfvc784-2LV-c`) platform. Synthesis was constrained with a 4 ns clock period in HLS and a 10 ns clock period for out-of-context Vivado implementation. All hardware metrics are extracted from post-route reports, as no on-board measurements were performed.

B. Experiments and Baselines

All experiments are reproducible from the public repository [16]: the reference model, the test benches, and the datasets are generated deterministically from a fixed seed. The evaluation uses five 8-bit grayscale 256×256 images spanning the range of spatial correlation: a constant field, a linear ramp, a piecewise-constant image with sharp edges, a $1/f$ surface with spatial correlation similar to natural images, and white noise as a control. The transform is applied over one-dimensional

$N = 8$ strips, matching the hardware; a separable two-dimensional variant (row then column) is also evaluated as a software-only extension.

The proposed architecture is contrasted against a baseline consisting of a standard lossless WHT realized with multipliers (a standard lifting/ladder mapping), synthesized on the same target to isolate the DSP savings attributable to the multiplier-free core.

C. Evaluation Metrics

Two metric axes were collected (Table II). Hardware efficiency was measured by look-up table (LUT), flip-flop (FF), and DSP-block utilization, the maximum operating frequency ($f_{\max}$), processing latency, and throughput in operations per second. Compression behavior was assessed by bit-exact lossless verification against the golden model and by the per-coefficient entropy reduction in bits per symbol.

TABLE II
EVALUATION METRICS

Axis	Metrics
Hardware cost	LUT, FF, DSP, $f_{\max}$, latency, throughput (OP/s)
Compression	bit-exact verification, entropy reduction (bits/pixel)

D. Functional Verification

Three properties were checked, and we state for each one what it does and does not cover.

First, *forward equivalence* between the reference model and the synthesizable core: over a fixed test vector and 10^5 random 8-bit blocks, both produce identical coefficients, with no divergence. Second, *inverse equivalence*: the reference inverse and the synthesizable inverse core agree over a fixed vector and 10^5 random coefficient vectors drawn from the full signed 16-bit range, a wider stimulus than the forward test uses. Both checks link the synthesizable sources directly, so a defect in either core would be caught. Third, *losslessness*: cascading the two synthesizable cores recovers the input exactly over 4×10^5 blocks, including the full signed 16-bit range. This check is separate from the equivalence tests and necessary: the forward core is exercised on 8-bit inputs, whose coefficients occupy a small region of the 16-bit space, while the inverse core is exercised on uniform 16-bit vectors, so neither equivalence test covers the composition. An equivalent check on the reference model confirms that the lifting construction itself is reversible.

Regarding dynamic range, an 8-bit input cannot overflow the 16-bit datapath: exhaustive evaluation over the extreme inputs $\{0, 255\}^8$ gives a largest coefficient magnitude of 1020, well inside the range of a signed 16-bit word. The same bound holds for $N = 16$ (2040), so the datapath width is not a constraint on larger block sizes.

Two limits of this verification should be stated explicitly. It is *source-level* equivalence, obtained by compiling the synthesizable sources with a host compiler: no C/RTL cosimulation was performed, so the generated register-transfer description itself is not covered. And the reference model

and the core implement the lifting butterfly with the same two expressions, which means these checks detect indexing and stage-ordering errors — as the inverse-equivalence check does by construction — but would not detect an error in the shared formula. The SystemC wrapper is a transaction-level driver around the same reference functions and exercises 2001 cases; the large-sample evidence above comes from the C++ test benches.

E. Entropy Reduction

Since the accelerator implements the transform stage only, its benefit is measured as the reduction in zeroth-order entropy (bits per symbol) between the pixels and the transform coefficients. Entropy is the lower bound on the rate of any zeroth-order lossless coder, so a lower value indicates more compressible data. The length-8 transform maps each block to one DC (mean) coefficient and seven detail coefficients of increasing sequency, which we treat as eight frequency bands. We report a pooled estimate (all coefficients in one histogram) and a per-band estimate (one entropy per frequency position, averaged), the latter being closer to the cost of a coder that models each band separately. Table III lists the results.

TABLE III
ZEROTH-ORDER ENTROPY (BITS/SYMBOL): PIXELS VS. WHT COEFFICIENTS.

Dataset	H_{pix}	1D ($N=8$, HW)		2D separable	
		pool	band	pool	band
Flat	0.00	0.54	0.00	0.12	0.00
Ramp	8.00	2.63	0.63	0.54	0.08
Edges	2.00	0.79	0.25	0.15	0.03
Natural	7.30	6.55	5.81	5.86	5.31
Noise	8.00	8.75	8.20	8.77	7.84

The reduction tracks the spatial correlation of the input. Smooth images (ramp, edges) concentrate almost all information in the DC band and drive the detail bands to zero entropy, giving per-band reductions of 87–92%. For the natural-like image the per-band reduction is 20% in one dimension and 27% for the separable two-dimensional transform; in absolute terms the second dimension adds 0.50 bits/symbol on top of the 1.50 that the one-dimensional transform already recovers. Fig. 2 shows both effects on that image. White noise, having no spatial correlation, is not improved by the transform and serves as a control; its coefficient entropy even exceeds the 8 bits of the input because the coefficients span a wider dynamic range than the pixels, which is also why the datapath is 16 bits wide. The gap between the one- and two-dimensional results motivates a two-dimensional core as future work.

F. Hardware Synthesis Results

Table IV summarizes the implemented results obtained from the Vitis HLS flow followed by post-route analysis in Vivado, considering the Kria KV260 as the target implementation platform. The design meets timing with positive slack and reaches an implemented frequency of 339.21 MHz. Resource usage remains moderate and consistent with a multiplier-free architecture, as indicated by zero DSP utilization.

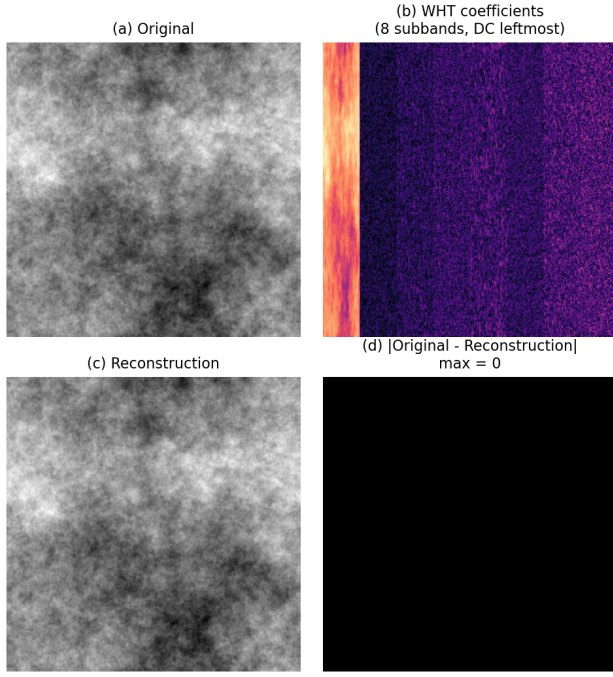


Fig. 2. Energy compaction and exact reconstruction on the natural-like image, using the same one-dimensional $N=8$ transform that the synthesized core implements. (b) regroups the coefficients by frequency band into eight vertical strips on a square-root magnitude scale: the leftmost strip is the DC band and retains a recognizable, horizontally compressed version of the image, while the seven detail bands are close to zero. That concentration is what the entropy reduction of Table III measures. (d) is identically zero, which is the lossless property.

TABLE IV
HLS / POST-ROUTE HARDWARE METRICS OF THE PROPOSED CORE.

Metric	Value
Critical data path delay (ns)	2.948
Implemented $f_{\max}$ (MHz)	339.21
WNS (ns)	6.604
LUT	2404
FF	2599
DSP	0
BRAM tile	4

These figures validate the practical viability of the proposed datapath for high-throughput operation, while preserving low arithmetic complexity and hardware efficiency.

A conventional C++ implementation of the forward Walsh–Hadamard Transform (FWHT) was also developed as a functional baseline for future architectural and resource comparisons. This baseline is independent of the bit-accurate SystemC golden model used to verify the reversible lifting-based HLS core.

Table V summarizes the verification results obtained from the C++ baseline implementation.

The computed coefficients matched the expected output, and the test bench reported a successful execution (PASS), confirming the functional correctness of the baseline implementation for the evaluated input vector. The execution log generated during verification is available in the project

TABLE V
VERIFICATION RESULTS OF THE BASELINE FWHT IMPLEMENTATION

Metric	Result
Input size	8 samples
Implementation	C++ baseline
Expected output	Matched
Test status	PASS

repository.

The C++ baseline serves as the functional reference for validating the proposed hardware implementation. Unlike the HLS accelerator, whose objective is to optimize hardware efficiency and throughput, the baseline focuses exclusively on numerical correctness. This separation allows the hardware implementation to be evaluated independently from algorithmic correctness before quantitative comparisons with related FPGA accelerators reported in the literature are performed during the final stage of the project.

As future work, the reported metrics will be compared quantitatively against integer-based Walsh–Hadamard and Discrete Cosine Transform accelerators from the literature to position the proposed solution in terms of frequency–area trade-offs.

DECLARATION ON THE USE OF GENERATIVE AI

During the preparation of this work, the authors used an AI-based assistant to consult concepts on the Walsh–Hadamard transform and lossless coding, to survey the state of the art and verify each cited source against its full text (author names, numerical results, and DOIs), to format the references in IEEE style and search for the index terms from the IEEE Thesaurus, and to review the manuscript critically for factual accuracy against the cited sources.

REFERENCES

- [1] A. Manjarres Garcia, C. Osorio Quero, J. Rangel-Magdaleno, J. Martinez-Carranza, and D. Durini Romero, “Parallel-pipeline fast Walsh-Hadamard transform implementation using HLS,” in *Proc. Int. Conf. Field-Programmable Technology (ICFPT)*, 2021, doi: 10.1109/ICFPT52863.2021.9609874.
- [2] H.-Y. Jung, R. Prost, and T.-Y. Choi, “A unified mathematical form of the Walsh–Hadamard transform for lossless image data compression,” *Signal Process.*, vol. 63, pp. 35–43, 1997, doi: 10.1016/S0165-1684(97)00138-2.
- [3] K. Komatsu and K. Sezaki, “Lossless 2D discrete Walsh-Hadamard transform,” in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2001, doi: 10.1109/ICASSP.2001.941320.
- [4] S. Hafizullah, M. S. S. V. S. Manideep, V. Sharma, P. K. Nath, A. Naugarhiya, and S. Verma, “An efficient hardware implementation of Walsh Hadamard transform for JPEG XR,” in *Proc. 15th IEEE India Council Int. Conf. (INDICON)*, 2018, doi: 10.1109/INDICON45594.2018.8987180.
- [5] Ž. Regoršek, A. Gorkič, and A. Trost, “Parallel lossless compression of raw Bayer images on FPGA-based high-speed camera,” *Sensors*, vol. 24, no. 20, art. 6632, 2024, doi: 10.3390/s24206632.
- [6] R. Ghodhbani, T. Saidani, L. Horigue, A. M. Algarni, and M. Alshammari, “An FPGA accelerator for real-time hyperspectral image compression based on JPEG2000 standard,” *Eng. Technol. Appl. Sci. Res.*, vol. 14, no. 2, pp. 13118–13123, 2024, doi: 10.48084/etasr.6853.
- [7] T. Alonso, G. Sutter, and J. E. López de Vergara, “LOCO-ANS: An optimization of JPEG-LS using an efficient and low-complexity coder based on ANS,” *IEEE Access*, vol. 9, pp. 106606–106626, 2021, doi: 10.1109/ACCESS.2021.3100747.

- [8] D. Nayak, K. Ray, T. Kar, and C. Kwan, "Walsh–Hadamard kernel feature-based image compression using DCT with bi-level quantization," *Computers*, vol. 11, no. 7, art. 110, 2022, doi: 10.3390/computers11070110.
- [9] C. Weng, X. Gu, and H. Jin, "Coded excitation for ultrasonic testing: A review," *Sensors*, vol. 24, no. 7, art. 2167, 2024, doi: 10.3390/s24072167.
- [10] A. Kobori, R. Takahashi, and M. Nakanishi, "A hardware architecture for the Walsh–Hadamard transform toward fast simulation of quantum algorithms," *CCF Trans. High Perform. Comput.*, vol. 2, pp. 211–220, 2020, doi: 10.1007/s42514-020-00028-7.
- [11] A. Amira and S. Chandrasekaran, "Power modeling and efficient FPGA implementation of FHT for signal processing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 15, no. 3, pp. 286–295, 2007, doi: 10.1109/TVLSI.2007.893606.
- [12] G. Bi, A. Aung, and B. P. Ng, "Pipelined hardware structure for sequency-ordered complex Hadamard transform," *IEEE Signal Process. Lett.*, vol. 15, pp. 401–404, 2008, doi: 10.1109/LSP.2008.922515.
- [13] J. Liu, Q. Xing, X. Yin, X. Mao, and F. Yu, "Pipelined architecture for a radix-2 fast Walsh–Hadamard–Fourier transform algorithm," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 62, no. 11, pp. 1083–1087, 2015, doi: 10.1109/TCSII.2015.2456371.
- [14] E. Wold and A. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Trans. Comput.*, vol. C-33, no. 5, pp. 414–426, 1984, doi: 10.1109/TC.1984.1676458.
- [15] A. T. Ho, J. Shen, A. K. Chow, and J. Woon, "Robust digital image-in-image watermarking algorithm using the fast Hadamard transform," in *Proc. Int. Symp. Circuits Syst. (ISCAS)*, 2003, doi: 10.1109/IS-CAS.2003.1205147.
- [16] MP-6160 Grupo 5, "WHT Lossless Accelerator Repository." [Online]. Available: https://github.com/MP6160-Grupo-5/MP6160_grupo_5